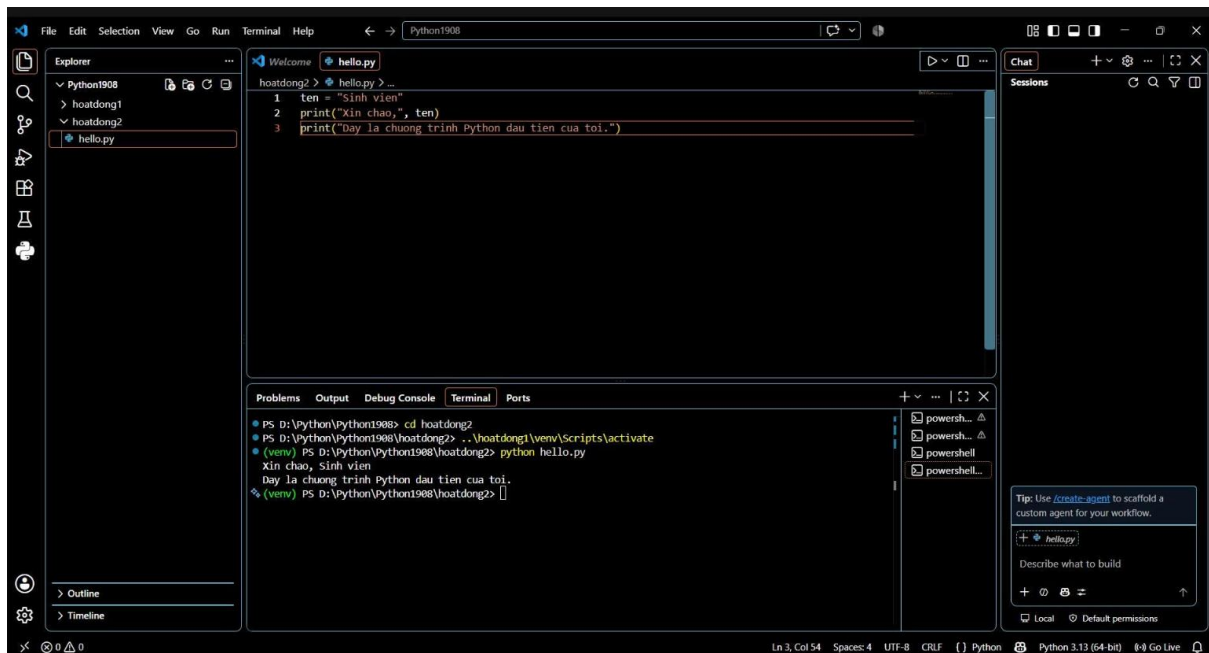
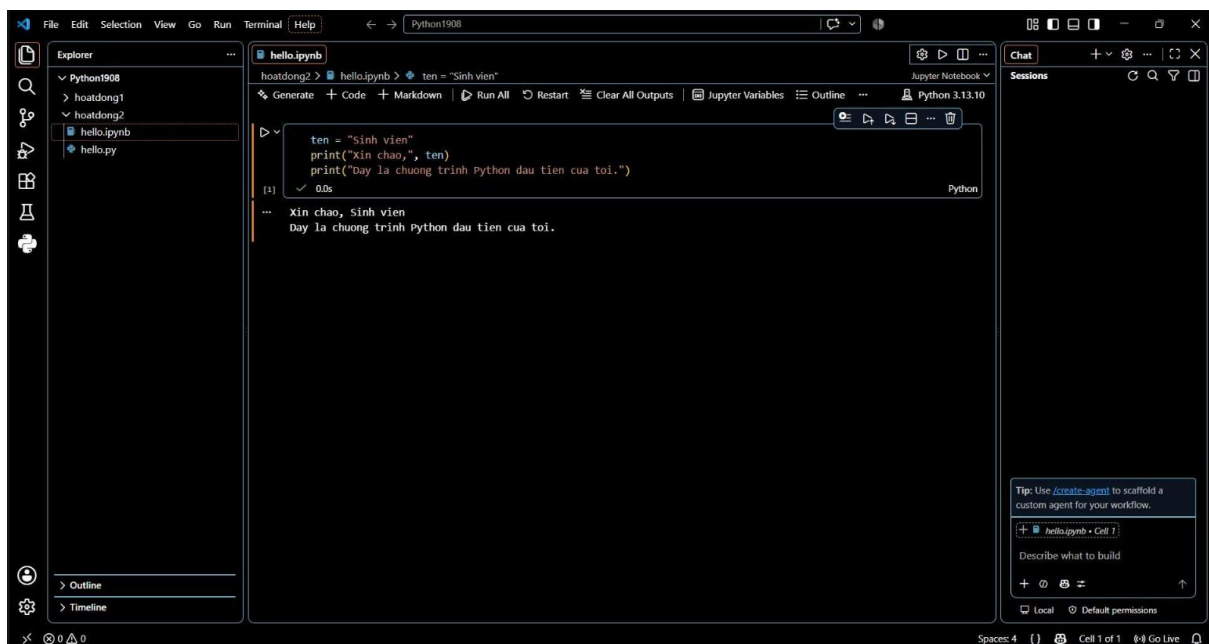


Sản phẩm 1

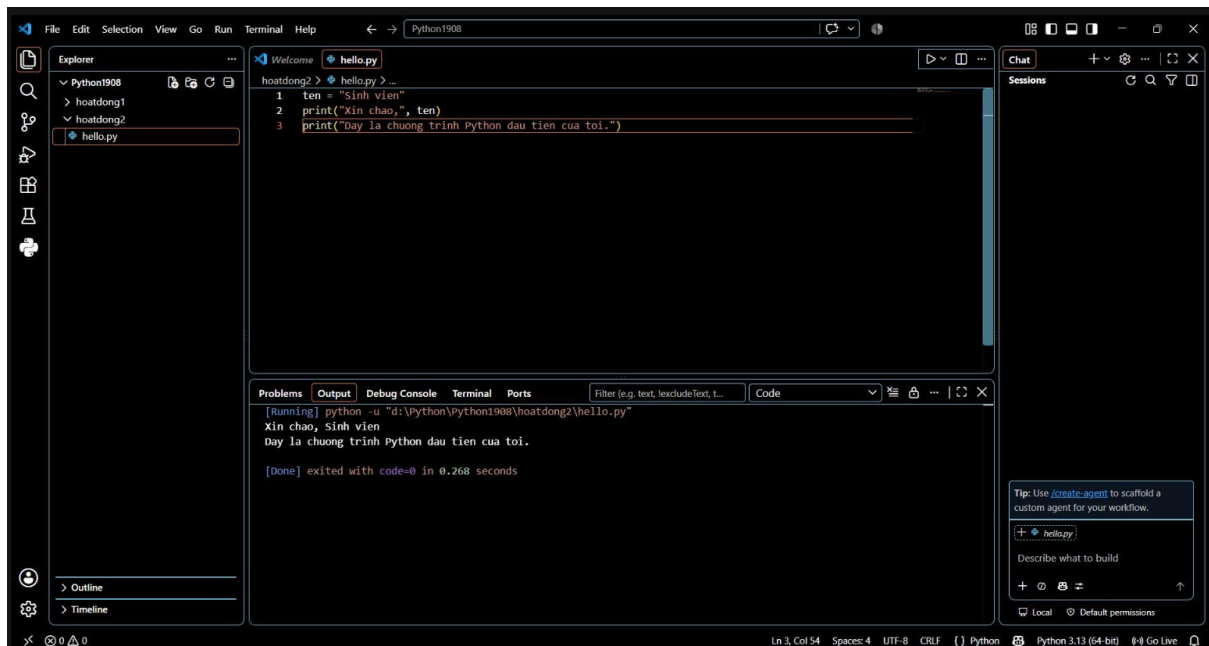
Cách 1:



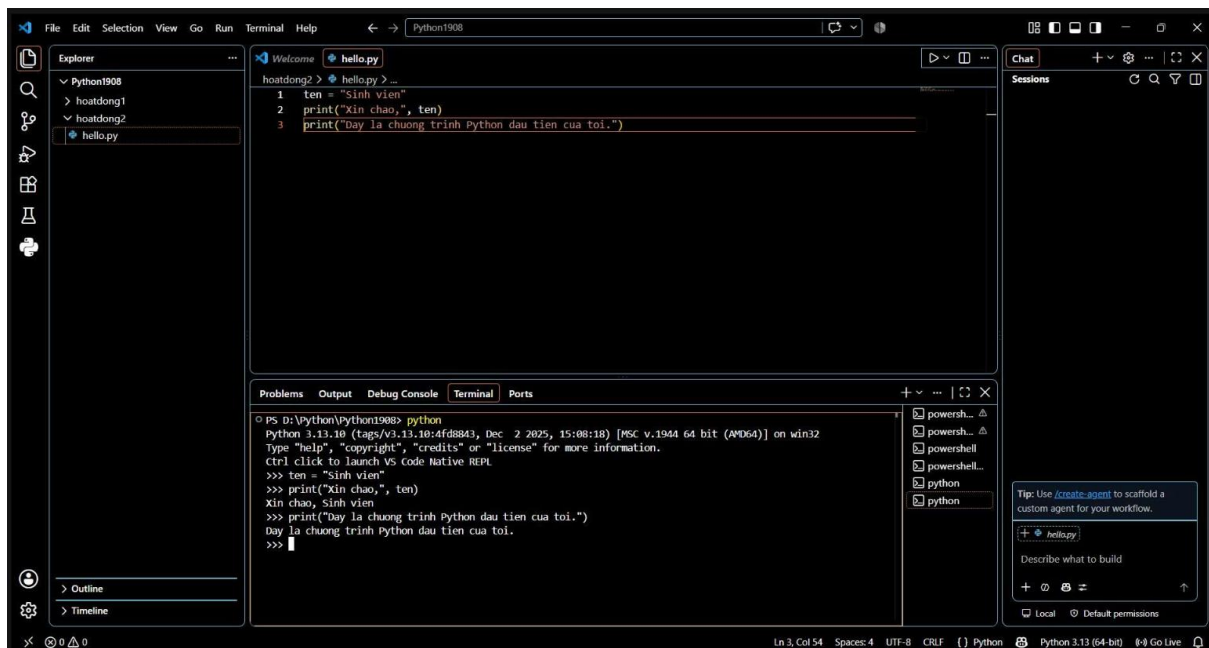
Cách 2:



Cách 3:



Cách 4:



Sản phẩm 2

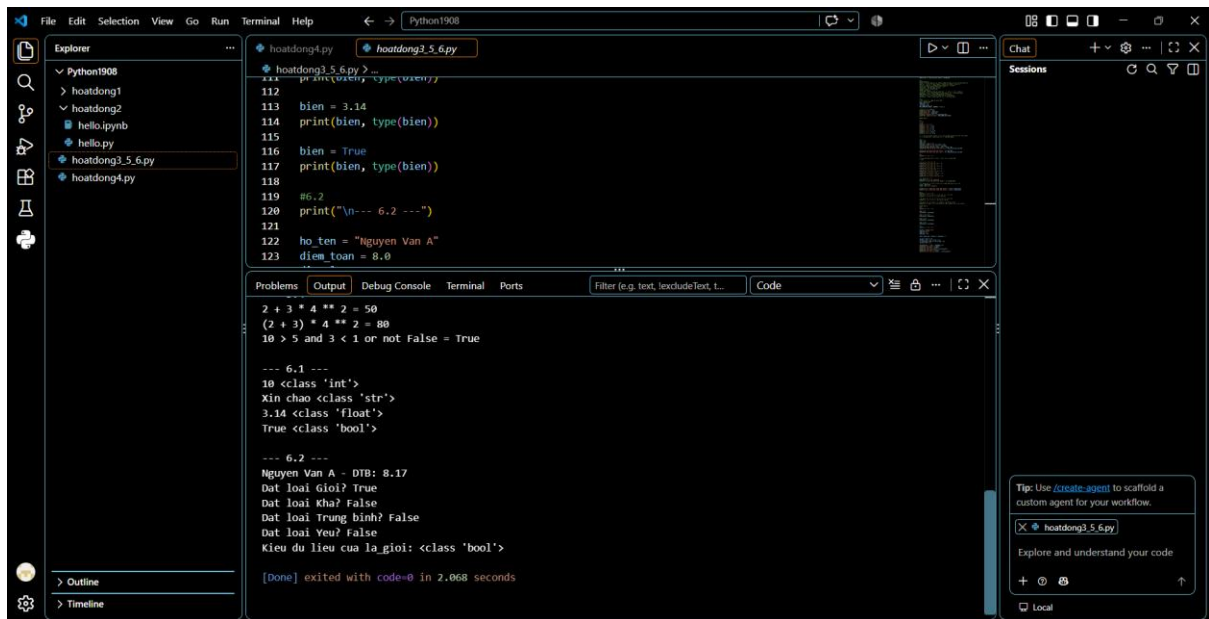
```
111 print(x, type(x))
112
113 bien = 3.14
114 print(bien, type(bien))
115
116 bien = True
117 print(bien, type(bien))
118
119 #6.2
120 print("\n--- 6.2 ---")
121
122 ho_ten = "Nguyen Van A"
123 diem_toan = 8.0
...
```

[Running] python -u "d:\Python\Python1908\hoatdong3_5_6.py"

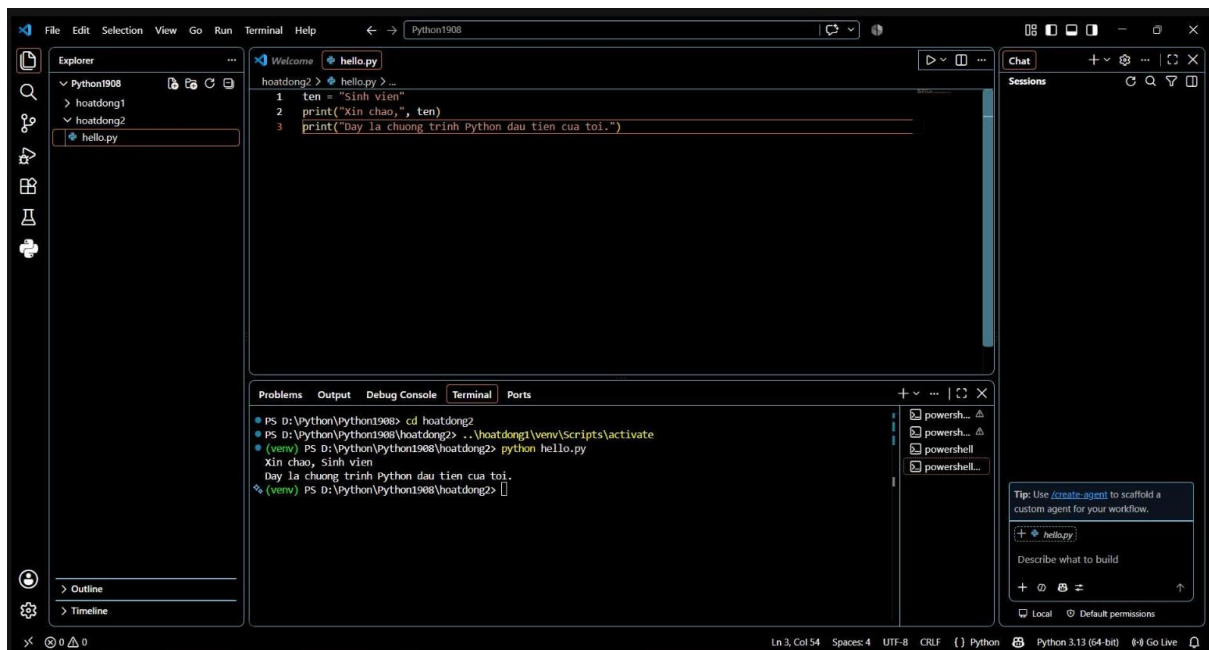
Ho va ten: Nguyen Van A
Diem Toan: 8.5
Diem Van: 7.0
So luong mon hoc: 2
Muc luong toi thieu: 5000000
--- 5.1 ---
a + b = 22
a - b = 12
a * b = 85
a / b = 3.4
a // b = 3
a % b = 2
a ** b = 1419857
--- 5.2 ---
Diem dat loai kha (and): True
Tuoi chua du 18 hoac tren 60 (or): False
Phu dinh dieu kien diem (not): False
Phu dinh dieu kien tuoi (not): True
--

```
112
113 bien = 3.14
114 print(bien, type(bien))
115
116 bien = True
117 print(bien, type(bien))
118
119 #6.2
120 print("\n--- 6.2 ---")
121
122 ho_ten = "Nguyen Van A"
123 diem_toan = 8.0
...
```

--- 5.3 ---
Gia tri x sau x += 5 la: 15
Gia tri x sau x -= 3 la: 12
Gia tri x sau x *= 2 la: 24
Gia tri x sau x /= 4 la: 6.0
Gia tri x sau x // 2 la: 3.0
Gia tri x sau x **= 3 la: 27.0
3 co nam trong danh_sach khong?: True
list1 is danh_sach (cung tham chieu): True
list2 is danh_sach (khac tham chieu): False
--- 5.4 ---
2 + 3 * 4 ** 2 = 50
(2 + 3) * 4 ** 2 = 80
10 > 5 and 3 < 1 or not False = True
--- 6.1 ---
10 <class 'int'>
Xin chao <class 'str'>
3.14 <class 'float'>
True <class 'bool'>



Sản phẩm 3



Sản phẩm 4

Hoạt động 2:

***Sự khác biệt giữa chạy file .py và gõ trực tiếp trong REPL là gì?**

Chạy file .py:

- Code được lưu lại vĩnh viễn trong file để tái sử dụng hoặc chia sẻ.

- Chương trình chạy toàn bộ từ trên xuống dưới trong một lần thi hành.
- Phải dùng hàm print() mới hiển thị được kết quả ra màn hình.
- Phù hợp để viết ứng dụng, phần mềm hoàn chỉnh hoặc các bài tập lớn.

Gõ trực tiếp trong REPL (Terminal):

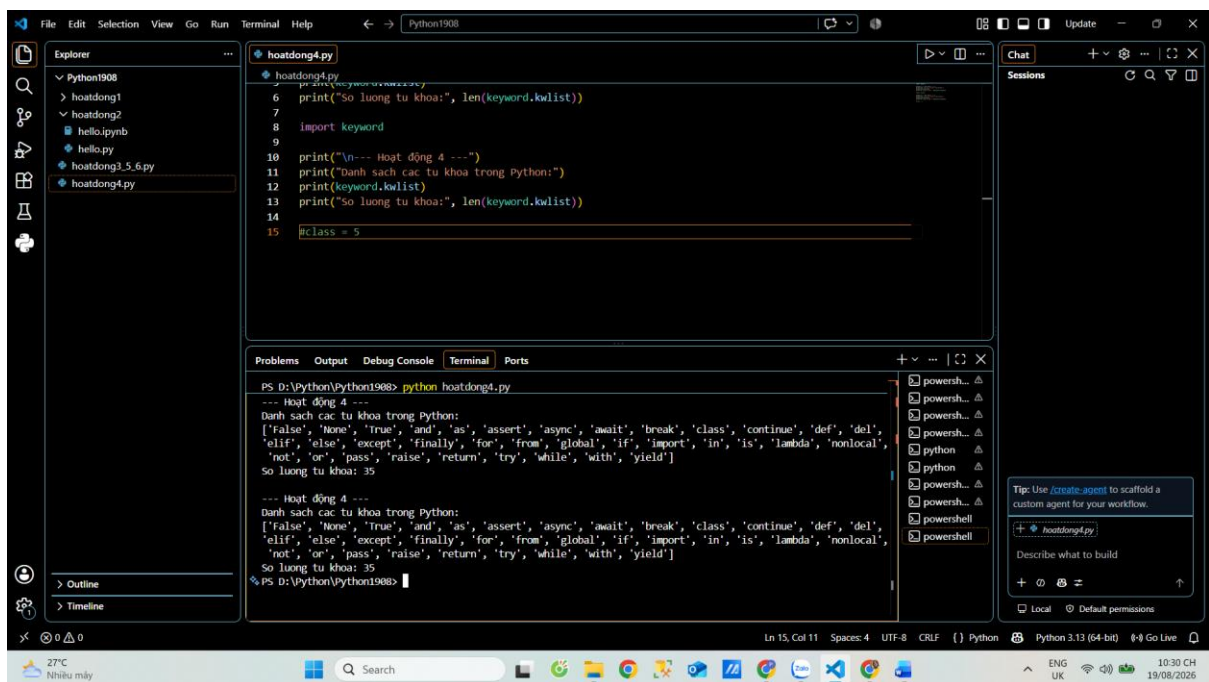
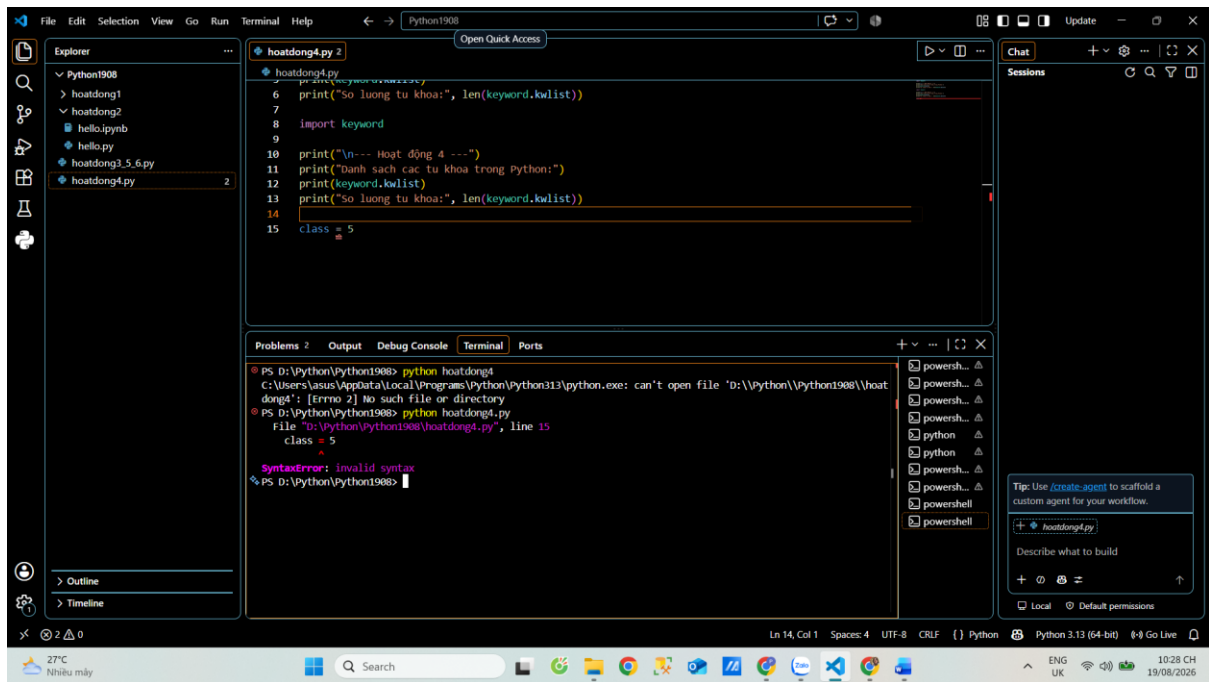
- Code chạy từng dòng ngay khi nhấn Enter và không tự động lưu lại sau khi đóng Terminal
- Tự động in kết quả của biểu thức đơn mà không cần dùng hàm print().
- Phù hợp để kiểm tra nhanh câu lệnh, thử nghiệm thuật toán hoặc debug lỗi ngắn.

***Khi nào nên dùng Jupyter Notebook thay vì file .py thông thường?**

Nên dùng Jupyter Notebook (.ipynb) trong các trường hợp:

- Chạy thử từng phần code (Cell): Khi muốn chạy từng đoạn code nhỏ để kiểm tra dữ liệu mà không cần phải thực thi lại toàn bộ chương trình từ đầu.
- Phân tích dữ liệu & Học máy (Data Science/AI): Cần quan sát ngay kết quả trực quan như bảng biểu, đồ thị, hình ảnh ngay dưới từng ô code.
- Làm báo cáo, tài liệu học tập: Khi cần kết hợp giữa code Python, ghi chú bằng chữ formatted (Markdown), công thức toán học và biểu đồ kết quả trong cùng một tài liệu duy nhất.

Hoạt động 4:



*Quan sát lỗi SyntaxError khi đặt class = 5 và giải thích:

- Thông báo lỗi trả về: SyntaxError: invalid syntax
- Giải thích: class là một từ khóa bảo lưu (reserved keyword) dùng để định nghĩa lớp trong lập trình hướng đối tượng. Trình biên dịch của Python đã mặc định quy tắc cú pháp cho các từ khóa này, do đó bạn không thể dùng chúng làm tên biến (định danh thông thường).

***Tại sao True, False, None cũng là từ khóa chứ không phải định danh thông thường?**

- Giá trị cố định của ngôn ngữ: True (Đúng) và False (Sai) đại diện cho các giá trị hằng số kiểu Boolean, còn None đại diện cho giá trị rỗng/không tồn tại.
- Đảm bảo tính bất biến (Immutability): Việc biến chúng thành từ khóa bảo lưu giúp ngăn chặn người dùng vô tình hay cố ý ghi đè/gán lại giá trị cho chúng (ví dụ: cấm làm thao tác `True = 0`). Điều này bảo vệ tính đúng đắn cho mọi logic so sánh và điều kiện trong toàn bộ chương trình Python.

Hoạt động 6:

***Vì sao cùng một biến có thể mang nhiều kiểu dữ liệu khác nhau trong Python?**

- Cơ chế Kiểu dữ liệu động (Dynamic Typing): Python tự động xác định kiểu dữ liệu của biến dựa trên giá trị được gán tại thời điểm chương trình thực thi (runtime) mà không cần khai báo trước kiểu.
- Bản chất biến là nhãn tham chiếu: Trong Python, biến không phải là "ô chứa" trực tiếp lưu giá trị, mà chỉ đóng vai trò là một nhãn dán (con trỏ/tham chiếu) trỏ tới đối tượng giá trị nằm trong bộ nhớ.
- Kiểu dữ liệu thuộc về đối tượng: Bản thân tên biến không có kiểu dữ liệu cố định. Khi bạn gán `bien = "Xin chào"`, tên biến chỉ đơn giản là bỏ trỏ tới đối tượng số nguyên 10 để chuyển sang trỏ tới đối tượng chuỗi "Xin chào".

***Điều này khác gì so with khai báo biến trong C/C++/Java?**

- C/C++/Java (Static Typing - Kiểu tĩnh):

Phải khai báo cố định kiểu dữ liệu trước khi sử dụng (ví dụ: `int bien = 10;`).

Tên biến gắn liền với một ô nhớ có kiểu cố định. Bạn không thể gán một giá trị chuỗi hay số thực vào biến đã khai báo kiểu `int` (chương trình sẽ báo lỗi ngay khi biên dịch).

- Python (Dynamic Typing - Kiểu động):

Không cần và không cho phép khai báo kiểu dữ liệu trước cho biến.

Một tên biến có thể thoải mái gán đi gán lại các giá trị thuộc bất kỳ kiểu dữ liệu nào ($\text{int} \rightarrow \text{str} \rightarrow \text{float} \rightarrow \text{bool}$) trong suốt quá trình chạy chương trình.